

Faculty of Engineering and Technology

Computer Science Department

Software Engineering - COMP433

Tea Shop

Software Requirement Specifications and Design-SRSD

Prepared by:

Project manager: Yasmin Itawi **ID:** 1221426

Requirement Engineer: Reema Qashoo **ID:** 1221009

Programmer: Suhaib Ewidat **ID:** 1230290

Instructor: Mohammed Husseinali

Group Number: 13

Sec: 1

Date: June 30, 2026

Table of Contents

1. Project Planning and Management 1
1.1 Names of Editors/Writers of the Report 1
1.2 Business Title 1
1.3 Group Name 1
1.4 Name of Students/Members 1
1.5 Role of Each Member 1
1.6 Project Management Strategy 1
1.7 Project Manager Report 2
1.8 Group Members Report 3
1.9 Group Members Images 4

2. Requirement Elicitation, Analysis and Modelling 5
2.1 Requirement statement-Business description 5
2.2 User and System Requirements - Modified 7
2.3 Effort and Time Estimation - Modified 10
2.4 Actor Analysis - Modified 13
2.5 Use Case Diagram – Modified 17
2.6 Task: Use Case Specifications 18
2.7 Activity Diagram 27

3. System Analysis and Modelling 28
3.1 System CLASS Diagrams 28
3.2 System Sequence modelling and Analysis 30
3.2.1 UC-12: Construct Custom Tea Blend [Writer: Suhaib Ewidat] 30
3.2.2 UC-18: Execute Order Checkout [Writer: Yasmin Itawi] 31
3.2.3 UC-19: Process Financial Transaction [Writer: Reema Qashoo] 32

4. System Design and Modelling 33
4.1 System Design Goals 33
4.1.1 Low Coupling and High Cohesion 33
4.1.2 Extensibility and Modularity 33
4.1.3 Specific System Design Goal: Transactional Data Integrity and Reliability 34
4.2 Component Diagram 35
4.3 Overall Architecture Diagram 36
4.3.1 Architecture Justification 37
4.3.2 Rationale for Choosing 3-Tier Layered Architecture 37
4.4 Deployment Diagram 39

Appendix-I: Project Meetings 40
Meeting 1 — Internal Team Meeting 40
Meeting 2 — Customer Meeting 40
Meeting 3 — Customer Meeting 41
Meeting 4 — Internal Team Meeting 41
Meeting 5 — Internal Team Meeting 42

Table of Figures

Figure 1: Use-Case Diagram 17
Figure 2:Overall Activity Diagram 27
Figure 3: Analysis Class Model 28
Figure 4: Detailed Class Model 29
Figure 5: Sequence Diagram [Suhaib] 30
Figure 6: Sequence Diagram [Yasmin] 31
Figure 7: Sequence Diagram [Reema] 32
Figure 8: Component Diagram 35
Figure 9: Overall Architecture Diagram 36
Figure 10: Deployment Diagram 39

1. Project Planning and Management

1.1 Names of Editors/Writers of the Report

Reema Qashoo, Yasmin Itawi, Suhaib Ewidat

1.2 Business Title

Tea Shop

1.3 Group Name

Group G13

1.4 Name of Students/Members

- Reema Qashoo 1221009
- Yasmin Itawi 1221426
- Suhaib Ewidat 1230290

1.5 Role of Each Member

- Project manager: Yasmin Itawi
- Requirement Engineer: Reema Qashoo
- Programmer: Suhaib Ewidat

1.6 Project Management Strategy

Our team followed a hybrid software process model that combines both Waterfall and Agile approaches. We worked in clear project phases such as requirements, analysis, and design, which helped us stay organized and complete each stage before moving to the next. At the same time, we remained flexible and updated our work based on feedback and new ideas during development.

The team held online meetings once a week at the beginning of the project to discuss progress, assign tasks, and review completed work. As the project moved closer to deadlines and tasks became more complex, we increased the number of online meetings to improve coordination and communication.

Decision-making in our team was collaborative. We discussed different ideas openly, and final decisions were made through voting when there were different opinions. This approach allowed every team member to participate in important project decisions.

Lucidchart was used as a tool for designing and maintaining system diagrams for the project.

1.7 Project Manager Report

Effective teamwork was maintained through strong collaboration and continuous communication. Rather than assigning tasks to individual members at the outset, a collaborative working style was adopted. For every task, meetings were held to discuss requirements, explore possible solutions, and determine the best approach before dividing responsibilities into Lead, Discussing & Reviewing, and Finalising roles.

Contribution by Members:

- **Yasmin Itawi (Project Manager):** Coordinated the overall project progress, organized team meetings, tracked deadlines, and ensured that each phase was completed on time. She led the Effort and Time Estimation and Activity Modelling tasks, and contributed to the Architecture Diagram and System Class Diagrams. She also played an important role in reviewing deliverables, maintaining consistency across different sections, and making sure the final report was well-structured and aligned with project requirements.
- **Reema Qashoo (Requirement Engineer):** Led the requirements analysis and modelling tasks, ensuring that the system requirements were clearly defined and accurately documented. She led the System Requirements and Use-Case Modelling sections, and contributed to the System Design Goals and Deployment Diagram. She also helped review and refine the modelling artefacts to ensure consistency between the written requirements and system design.
- **Suhaib Ewidat (Programmer):** Focused mainly on the technical and implementation-oriented aspects of the project. He led the User Requirements and Actor Analysis sections, and contributed to the Detailed Class Diagram and Component Diagram. He played an important role in technical discussions, helping translate system requirements into practical design decisions and structured models.

Challenges:

One of the main challenges during the project was time management and dealing with deadlines. Another challenge was balancing the project workload with other university courses and exam periods, which sometimes made task coordination more difficult. In addition, some design tasks, especially diagrams and architecture-related work, required long discussions before reaching agreement. Creating and refining diagrams often took significant time and effort to ensure accuracy and consistency. Scheduling meetings also became challenging in some cases, since finding suitable meeting times for all team members was not always easy.

Project Performance Review:

Overall, the project was highly successful. The team showed strong commitment, collaboration, and continuous improvement throughout all phases. Responsibilities were distributed based on each member's role and strengths, while maintaining shared discussion and review to ensure a unified final outcome. The instructor's feedback after each phase proved highly beneficial and served as a catalyst for continuous improvement. This progression was clearly reflected in performance, as each phase achieved a higher grade than the previous one.

1.8 Group Members Report

Yasmin Itawi (Project Manager):

I believe our project was successful because we improved our work in every phase and benefited from the feedback we received. The teamwork and communication between us helped us complete the project effectively. My contribution included leading the Effort and Time Estimation task and the Activity Modelling section. I also contributed to the Architecture Diagram and System Class Diagrams, while participating in discussions, reviewing deliverables, organizing meetings, and helping ensure the final report was completed on time.

Reema Qashoo (Requirement Engineer):

I believe the project was successful because our team maintained strong collaboration and continuously improved the quality of our work. Through this project, I gained more experience in requirements analysis and system modelling. My contribution included leading the System Requirements and Use-Case Modelling sections I also contributed to the System Design Goals and Deployment Diagram, in addition to discussing, reviewing, and refining many project deliverables to ensure consistency between requirements and design.

Suhaib Ewidat (Programmer):

In my opinion, the project was successful because we worked together efficiently and supported each other during all project phases. The project helped me improve my technical and analytical skills. My contribution included leading the User Requirements and Actor Analysis sections. I also contributed to the Component Diagram, while actively participating in technical discussions and helping transform requirements into structured system models and implementation oriented design decisions.

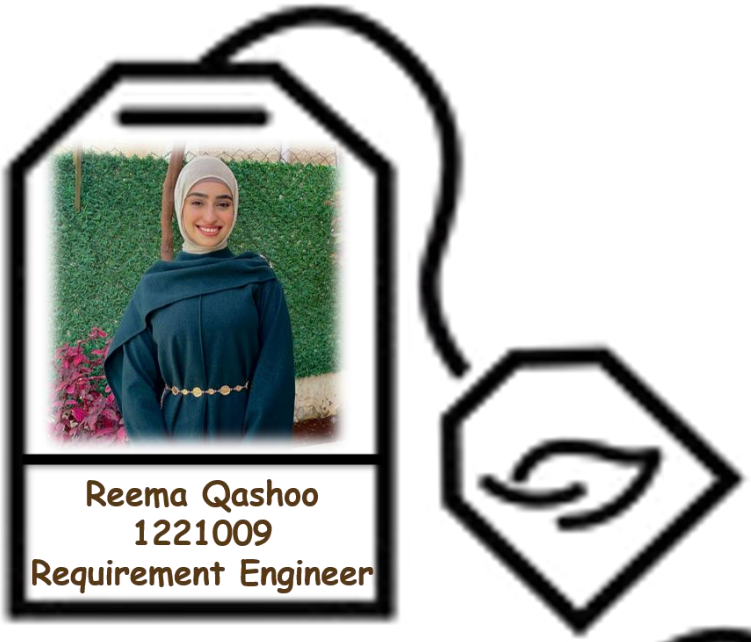
Note on Task Distribution:

Individual use cases and their corresponding sequence diagrams were assigned separately to team members based on task distribution. **UC-12: Construct Custom Tea Blend** was completed by Suhaib Ewidat, **UC-18: Execute Order Checkout** was completed by Yasmin Itawi, and **UC-19: Process Financial Transaction** was completed by Reema Qashoo.

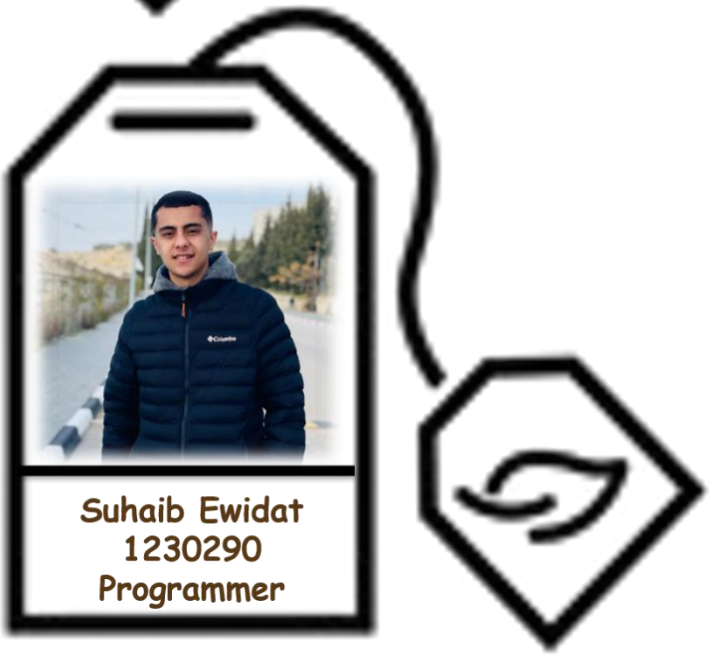
Group tasks, including the Use Case Diagram, Activity Diagram, Class Diagrams, and System Design Diagrams, were completed collaboratively. For each group task, one member took the lead while the other members helped through discussion, reviewing, and final improvements.

1.9 Group Members Images









2. Requirement Elicitation, Analysis and Modelling

2.1 Requirement statement-Business description

1. Overview

The Online Tea Shop is an online business that sells different types of tea through a website and a mobile application. It offers a wide range of products, including black tea, green tea, herbal tea, loose-leaf tea, tea bags, tea cans, and custom tea blends. The shop serves individual customers as well as businesses (B2B), making it suitable for everyday tea drinkers, health-conscious people, students, families, offices, and cafes.

Customers can browse products by category, brand, or flavor, search for specific items, and receive smart tea recommendations based on their preferences. They can also create their own custom tea blends, place orders online, choose a payment method, and have their orders delivered to their location.

2. Business Capacity

- Tea products: **30 to 45**
- Tea suppliers: **4 to 6 local and international suppliers**
- Monthly active customers: **600 to 1000 users**
- Daily orders: **15 to 30 orders per day**
- Monthly orders: **450 to 900 orders per month**
- Average items per order: **2 to 3 products**
- Monthly sales: **1000 to 2500 individual products**
- Active B2B clients: **3 to 5**

3. Services Offered

- Customer registration and personal accounts
- Browse and search tea products by category, brand, or flavor
- Smart tea recommendations based on customer preferences
- Create custom tea blends using different tea bases and herbs
- Shopping cart and online checkout
- Payment by credit card, e-wallet, or cash on delivery
- Order confirmation through email or mobile app notifications
- Home delivery with real-time order tracking
- Gift boxes and seasonal tea collections
- Loyalty points, referral rewards, and birthday discounts
- Product ratings and customer reviews
- Wholesale (B2B) ordering with recurring deliveries and business contracts

4. Service Operation

Customers visit the Online Tea Shop through the website or mobile application. They can browse tea products by category, brand, or flavor, or use the smart recommendation feature to find teas that match their preferences. Customers can also create their own custom tea blends before adding products to the shopping cart.

At checkout, customers enter their delivery address, choose their preferred payment method (credit/debit card, e-wallet, or cash on delivery), and confirm the order. A confirmation message is then sent through email or the mobile application.

The order is forwarded to the warehouse, where the staff collect, package, and prepare the products for delivery. A delivery partner delivers the order to the customer, who can track the order status in real time. After receiving the order, customers can rate the products, write reviews, or contact customer support if they have any questions or problems.

For business customers (B2B), the system also supports wholesale orders with scheduled deliveries based on business agreements.

5. Employees and Their Roles

- **Product Manager (1):** Manages the product catalog, suppliers, and product quality.
- **Order Processing Staff (3):** Prepare, package, and dispatch customer orders.
- **Platform/Tech Team (2):** Maintain the website and mobile application and manage payment integration.
- **Delivery Coordinator (1):** Coordinates with delivery partners and follows up on deliveries.
- **Customer Service Representatives (2):** Handle customer inquiries, complaints, and product reviews.
- **Marketing Specialist (1):** Manages social media, promotions, referral campaigns, and seasonal offers.
- **Finance Officer (1):** Handles payments, invoices, and B2B contracts.

2.2 User and System Requirements - Modified

User Requirements

- **Lead:** Suhaib Ewidat
- **Contributors:** Reema Qashoo(Discussing&Reviewing), Yasmin Itawi(Writing&Finalising)

UR1.0: A new user shall register a personal account.

UR2.0: A customer shall view detailed product information.

UR3.0: A customer shall search for tea products.

UR4.0: A customer shall filter search results.

UR5.0: A customer shall manage items in a shopping cart.

UR6.0: A customer shall place an order.

UR7.0: An authorized administrator shall manage product listings in the store catalog.

UR8.0: An authorized administrator shall manage stock inventory levels.

UR9.0: A customer shall track the delivery status of an order.

UR10.0: A customer shall submit a product review.

System Requirements

- **Lead:** Reema Qashoo
- **Contributors:** Yasmin Itawi(Discussing&Reviewing),Suhaib Ewidat(Writing&Finalising)

UR1.0: A new user shall register a personal account.

- **SR1.1**: The system shall require the submission of a unique email address, password, first name, and last name to create an account.
- **SR1.2**: The system shall validate that the submitted email address conforms to the standard format, for example, name@domain.com.
- **SR1.3**: The system shall reject registration requests for email addresses already existing in the user records.
- **SR1.4**: The system shall enforce password complexity of at least 8 characters, including uppercase, lowercase, and numeric characters.
- **SR1.5**: The system shall generate a verification link to the submitted email address upon registration.
- **SR1.6**: The system shall activate the account upon confirmation of ownership via the verification link.

UR2.0: A customer shall view detailed product information.

- **SR2.1**: The system shall retrieve product descriptions, ingredient lists, price points, and visual media when a specific item is selected.
- **SR2.2**: The system shall present active customer ratings and historical product reviews on the product detail interface.
- **SR2.3**: The system shall display the real-time availability status for the selected product.

UR3.0: A customer shall search for tea products.

- **SR3.1**: The system shall allow customers to search for tea products by entering one or more keywords.
- **SR3.2**: The system shall match search queries against product names, descriptions, and tea types.
- **SR3.3**: The system shall return all products that satisfy the entered search criteria.
- **SR3.4**: The system shall notify the customer when no products match the entered search criteria.
- **SR3.5**: The system shall return search results within 3 seconds under normal operating conditions.

UR4.0: A customer shall filter search results.

- **SR4.1**: The system shall apply tea type filters (e.g., green, black, herbal) to search results.
- **SR4.2**: The system shall apply price range filters to search results.
- **SR4.3**: The system shall apply customer rating filters to search results.
- **SR4.4**: The system shall process multiple concurrent filter parameters.
- **SR4.5**: The system shall retrieve only those products satisfying all active filter criteria.

UR5.0: A customer shall manage items in a shopping cart.

- **SR5.1**: The system shall allow customers to add tea products to a virtual shopping cart directly from product listings or details.
- **SR5.2**: The system shall allow customers to modify item quantities in the shopping cart.
- **SR5.3**: The system shall calculate the subtotal of all items in the shopping cart in real time.
- **SR5.4**: The system shall persist the shopping cart items across the user's active session.

U6.0: A customer shall place an order.

- **SR6.1**: The system shall allow customers to include one or more tea products or saved custom blends in a single order.
- **SR6.2**: The system shall verify that all items in the order are available in the requested quantities before confirming the order.
- **SR6.3**: The system shall calculate the total order price, including applicable taxes and delivery charges, before the customer confirms the order.
- **SR6.4**: The system shall require customers to provide a valid delivery address before an order is confirmed.

- **SR6.5**: The system shall generate a unique order reference number upon order confirmation.

UR7.0: An authorized administrator shall manage product listings in the store catalog.

- **SR7.1**: The system shall allow administrators to create new product profiles by entering a name, description, category, and unit price.
- **SR7.2**: The system shall allow administrators to edit descriptions, categories, and prices of existing product profiles.
- **SR7.3**: The system shall hide a product profile from public search visibility without deleting historical order records.
- **SR7.4**: The system shall restrict product catalog modifications exclusively to users authenticated with administrator credentials.

UR8.0: An authorized administrator shall manage stock inventory levels.

- **SR8.1**: The system shall allow administrators to adjust stock quantities for any product in the inventory database manually.
- **SR8.2**: The system shall log all manual inventory adjustments with a timestamp and the administrator's account identifier.
- **SR8.3**: The system shall generate an automatic warning notification to administrators when a product's stock falls below a predefined threshold.
- **SR8.4**: The system shall flag a product as out of stock automatically when its inventory count reaches zero.

UR9.0: A customer shall track the delivery status of an order.

- **SR9.1**: The system shall assign one of the following statuses to each confirmed order: Processing, Dispatched, Out for Delivery, or Delivered.
- **SR9.2**: The system shall allow customers to retrieve the current delivery status of any confirmed order using its reference number.
- **SR9.3**: The system shall allow authorized staff to update the delivery status of a confirmed order.
- **SR9.4**: The system shall record the date and time of each status change.
- **SR9.5**: The system shall provide an estimated delivery date for each dispatched order.

UR10.0: A customer shall submit a product review.

- **SR10.1**: The system shall accept review submissions exclusively for products associated with a "Delivered" order status in the customer's history.
- **SR10.2**: The system shall require a numerical rating between 1 and 5 and a written comment for each review.
- **SR10.3**: The system shall link each review to the specific product and customer account.
- **SR10.4**: The system shall restrict review submissions to one per purchased product.
- **SR10.5**: The system shall process edits or deletions of previously submitted reviews by the author.

2.3 Effort and Time Estimation - Modified

- **Lead:** Yasmin Itawi
- **Contributors:** Suhaib Ewidat(Discussing&Reviewing), Reema Qashoo(Writing&Finalising)

Method: Simplified person-week (pw) method. 1 pw = one developer working full-time for one week. A 30% buffer is added to account for testing, review, and unexpected delays. The team consists of **3 developers**.

Estimation Table

User Requirements	Estimated Effort	Estimated No. of Developers	Total Effort (for one developer)
UR1	1 pw	1	1 × 1 = **1 pw**
UR2	1 pw	1	1 × 1 = **1 pw**
UR3	2 pw	2	2 × 2 = **4 pw**
UR4	1 pw	1	1 × 1 = **1 pw**
UR5	2 pw	2	2 × 2 = **4 pw**
UR6	3 pw	3	3 × 3 = **9 pw**
UR7	2 pw	3	2 × 3 = **6 pw**
UR8	2 pw	2	2 × 2 = **4 pw**
UR9	1 pw	1	1 × 1 = **1 pw**
UR10	1 pw	1	1 × 1 = **1 pw**
Total effort / avg	**16 pw**	**17 ÷ 10 = 1.7 devs**	**32 pw**

Schedule time 30%	16 × 1.3 = 21 w (min time to complete)	32 × 1.3 = 42 w (max time to complete)
Cost	Avg salary = **$300**	300 × 42 = **$12,600**
Profit margin (min=10%, max=30%)	Min Cost	12,600 × 1.1 = **$13,860**
	Max Cost	12,600 × 1.3 = **$16,380**

Since there are 3 developers in the team, all of them will be utilized to optimize development time.

- Total effort ÷ number of developers = 32 ÷ 3 = **10.67 weeks**
- Basic time = 10.67 weeks
- **Schedule time 30%:** 10.67 × 1.30 = 13.87 w ≈ **14 w**
- **Cost = avg salary × number of developers × weeks ➔ 300 × 3 × 14 = $12,600**

Profit margin (min=10%, max=30%):

- Min Cost: 12,600 × 1.10 = **$13,860**
- Max Cost: 12,600 × 1.30 = **$16,380**

The optimized schedule of **14 weeks** fits within the semester timeframe, achieved by fully utilizing all 3 developers in parallel across the most complex requirements (UR6, UR7) and distributing the remaining work to avoid idle time.

❖ **Detailed Complexity & Effort Justification**

To make the development timeline realistic, the project requirements were divided into three levels of complexity: high, medium, and low. The classification is based on how difficult each requirement is, including backend logic, data handling, security, and system management.

1. High Complexity Requirements (6 pw – 9 pw)

UR6.0: Place Order (9 pw)

Justification: This is one of the most important parts of the Tea Shop system because it handles customer payments and order processing. It needs a lot of effort because the system must safely manage multiple orders at the same time. For example, it should prevent two customers from buying the last tea package together. The system also needs to calculate product prices, shipping cost, taxes, and coupon discounts in real time. After checkout, the shopping cart data must be converted into a saved order record with a unique tracking number.

UR7.0: Admin Catalog Management (6 pw)

Justification: This requirement needs strong security because only admins should be able to manage products. The system must separate admin access from regular customer access. It also needs to handle removed products carefully. Instead of fully deleting a tea product, the system should hide or archive it so old orders and sales records remain correct.

2. Medium Complexity Requirements (2 pw – 4 pw)

UR3.0: Search Products (4 pw)

Justification: Product searching is more than a simple database search. The system should allow flexible searching using keywords. It needs to check multiple product details such as product name, description, and tea category to return useful results.

UR5.0: Manage Shopping Cart (4 pw)

Justification: Managing a shopping cart requires the system to save user selections while they browse. The cart should not lose items if the user refreshes the page or disconnects for a short time. The system must also update total price and item quantity instantly.

UR8.0: Admin Inventory Management (4 pw)

Justification: Inventory management is not just changing stock numbers manually. The system should automatically update product quantities after purchases. When stock reaches zero, the product should be marked as "Out of Stock" or hidden. The system should also alert store managers when stock becomes low.

3. Low Complexity Requirements (1 pw each)

UR1.0: Account Registration (1 pw)

Justification: This is a simple form where users enter their information. The system only needs to validate inputs such as email format and password length before saving the data.

UR2.0: View Product Details (1 pw)

Justification: This is a simple read-only feature. The system gets a product ID from the user request and displays product information such as name, price, description, and image.

UR4.0: Filter Search Results (1 pw)

Justification: Filtering uses simple conditions like tea type or price range. Since it works on search results that were already found, it does not need much extra work.

UR9.0: Track Order Delivery (1 pw)

Justification: This is another simple read-only feature. The system uses the order tracking number to find the order status and display updates such as Processing, Shipped, or Delivered.

UR10.0: Submit Product Review (1 pw)

Justification: Before allowing a review, the system only needs to check if the user actually bought the product. This helps prevent fake or spam reviews.

2.4 Actor Analysis - Modified

- **Lead:** Suhaib Ewidat
- **Contributors:** Reema Qashoo (Discussing &Reviewing), Yasmin Itawi (Discussing &Finalising)

Actor	Role	Description
Customer	Primary	Represents a registered end-user of the Tea Shop system. Can browse and search tea products, view product details, manage a shopping cart, apply filters and coupons, view personalized recommendations, create and save custom tea blends, place orders, make payments, track delivery status via the external delivery service, manage their profile, and submit product reviews.
Store Admin	Primary	Represents a privileged internal staff member responsible for platform governance. Can manage the product catalogue (add, update, remove tea products), manage inventory stock levels, configure system settings, manage employee roles and access permissions, view sales and business intelligence reports, and manage customer accounts.
Warehouse Staff	Primary	Represents internal fulfilment personnel operating within the physical packaging facility. Can view pending order queues, verify ingredient availability, prepare and package standard and custom tea blend orders, generate shipping labels, and update order status to "Ready for Shipping" before handoff to the external delivery service.
Customer Support Rep	Primary	Represents front-line support personnel who handle post-purchase customer issues. Can review and respond to customer inquiries, process order modifications or cancellations, manage return and refund requests, and resolve delivery complaints through the support dashboard.
Marketing Specialist	Primary	Represents internal marketing personnel responsible for customer acquisition and retention. Can create and schedule promotional coupon codes, configure flash sales and seasonal discount campaigns with defined start and end dates, and manage customer loyalty reward milestones. Scheduled campaigns are activated and deactivated automatically by the System Timer.
Finance Officer	Primary	Represents the internal accountant responsible for fiscal oversight. Can review daily payment transaction records, audit order payment histories, and authorize monetary refunds for returned or cancelled orders through the financial management module.
Payment Gateway	Secondary	Represents an external financial service provider (PayPal or a card processor) integrated via API to securely handle payment authorization. Validates card details using tokenization and returns an approval or rejection response to the system. Does not store sensitive payment data on behalf of the Tea Shop.

Email Service	Secondary	Represents an external transactional email relay provider. Responds to system-triggered send requests to dispatch account verification links, password reset tokens, order confirmation receipts, and delivery status notifications to customers. Does not initiate any interaction independently.
Delivery Service	Secondary	Represents a third-party external logistics provider (as: Aramex) integrated via API and responsible for all order fulfilment and delivery operations. The system pushes shipping manifests to this actor once an order is marked "Ready for Shipping", and receives automated real-time tracking status updates (Dispatched, Out for Delivery, Delivered) via webhooks to update the customer facing tracking interface.

Use Case List

- **UC-01: Register Personal Account**
  A new customer submits personal details to create a verified system account.
- **UC-02: Authenticate User Login**
  A registered user enters security credentials to access their role-specific system dashboard.
- **UC-03: Process Password Recovery**
  A customer requests a secure identity verification link via email to reset a forgotten password.
- **UC-04: Maintain User Profile**
  A customer updates their personal contact information and account communication preferences.
- **UC-05: Monitor Customer Accounts**
  The Store Admin reviews customer account statuses and manages operational access restrictions.
- **UC-06: Browse Tea Catalogue**
  A customer views the public list of available tea products, descriptions, and base ingredients.
- **UC-07: Filter Search Results**
  A customer narrows down product search results using advanced criteria such as price, type, or rating.
- **UC-08: View Product Specifications**
  A customer inspects detailed ingredient breakdowns, health benefits, and user reviews of a selected tea.
- **UC-09: Retrieve Personalized Recommendations**
  A customer views tailored tea suggestions generated dynamically based on their past purchasing behavior.
- **UC-10: Construct Custom Tea Blend**
  A customer selects raw organic ingredients and adjusts percentages to create a unique custom tea formula.
- **UC-11: Save Custom Formula**
  A customer saves a successfully configured custom tea blend directly to their profile for future orders.

- **UC-12: Update Recommendation Cache**
  The system automatically refreshes the background recommendation engine data at predefined time intervals.
- **UC-13: Maintain Product Catalogue**
  The Store Admin updates the active list of products, ingredient specifications, and public retail prices.
- **UC-14: Maintain Shopping Cart Items**
  A customer adds, removes, or modifies product quantities within their active digital shopping cart.
- **UC-15: Apply Promotional Discount**
  A customer enters an active coupon code during checkout to deduct rewards from the order subtotal.
- **UC-16: Execute Order Checkout**
  A customer confirms their final cart contents, inputs delivery coordinates, and submits a binding order request.
- **UC-17: Process Financial Transaction**
  The system handles electronic credit card or digital wallet verification securely via an external payment gateway.
- **UC-18: Track Shipment Progress**
  A customer monitors real-time geolocation delivery milestones provided by the external logistics service webhook.
- **UC-19: Submit Product Feedback**
  A customer rates and writes an official review for a specific tea product after successful order delivery.
- **UC-20: Handle Payment Rejection**
  The system manages transaction exception flows when the payment gateway returns a declined status code.
- **UC-21: Review Pending Order Queue**
  Warehouse staff views the real-time chronological list of paid customer orders awaiting physical preparation.
- **UC-22: Verify Raw Ingredient Availability**
  Warehouse staff cross-checks required item quantities against current physical shelf inventory before assembly.
- **UC-23: Execute Product Packaging**
  Warehouse staff measures ingredients, prepares custom formulas, and packs the physical items into standard containers.
- **UC-24: Dispatch Shipping Manifest**
  The system automatically pushes packed order metadata and dimensional data to the external delivery service API.
- **UC-25: Update Order Fulfillment Status**
  Warehouse staff changes the internal order tracking database flag to "Ready for Shipping" upon successful packing.
- **UC-26: Optimize Inventory Stock Levels**
  The Store Admin reviews master ingredient volumes and inputs restock batches received from loose leaf suppliers.

- **UC-27: Configure Promotional Campaigns**
  The Marketing Specialist designs seasonal discount campaigns, sets validation boundaries, and generates new coupon codes.
- **UC-28: Maintain Loyalty Programme Criteria**
  The Marketing Specialist defines customer reward point distribution formulas and customer milestone achievement tiers.
- **UC-29: Manage Operational Support Tickets**
  A Customer Support Representative reviews, responds to, and archives customer inquiries submitted via help forms.
- **UC-30: Handle Order Discrepancies**
  A Customer Support Representative modifies address details or resolves issues on orders prior to warehouse processing.
- **UC-31: Manage Order Returns**
  A Customer Support Representative evaluates boundary conditions to approve or reject customer-submitted return requests.
- **UC-32: Audit Financial Transaction Records**
  The Finance Officer reviews daily digital balance sheets, settlement tallies, and master sales ledger histories.
- **UC-33: Authorize Monetary Refund**
  The Finance Officer triggers a programmatic refund call to the external payment gateway for validated order cancellations.

2.5 Use Case Diagram – Modified

- **Lead:** Reema Qashoo
- **Contributors:** Yasmin Itawi (Discussing &Reviewing), Suhaib Ewidat (Discussing &Finalising)

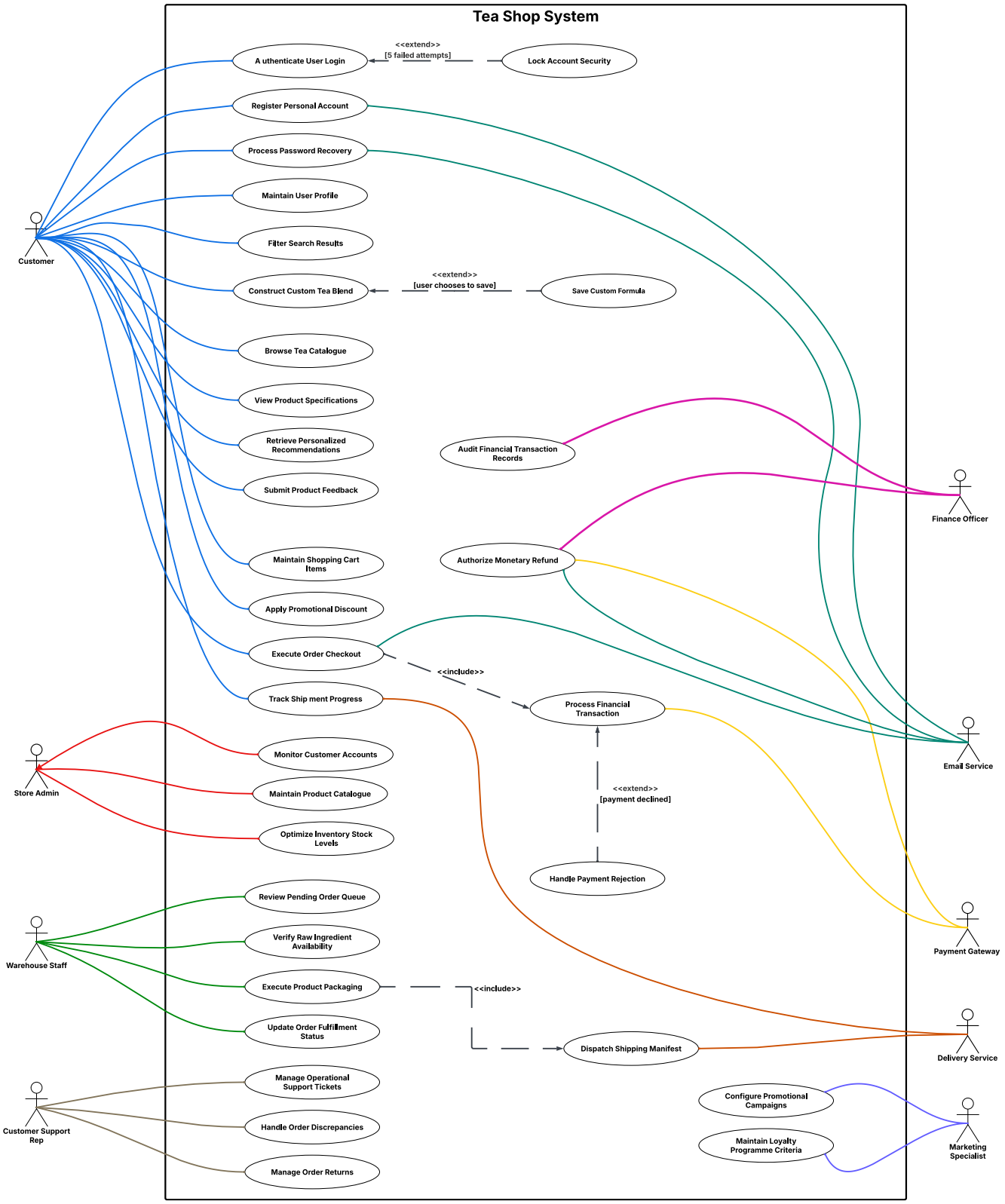


Figure 1: Use-Case Diagram

2.6 Task: Use Case Specifications

UC-12: Construct Custom Tea Blend [Writer: Suhaib Ewidat]

Use Case Title	UC-12: Construct Custom Tea Blend
Description	This use case describes how a registered Customer selects raw organic ingredients and adjusts percentages to create a unique custom tea formula. The system validates ingredient availability, calculates the final price, and saves the successfully configured formula directly to the Customers profile for future orders
Actors	Primary Actor: Customer
Data	Base tea types, ingredient options, ingredient quantities, unique blend ID, total calculated price
Stimulus / Trigger	The Customer selects the "Construct Custom Blend" option from the catalog interface.
Pre-conditions	1. The Customer is registered and successfully logged in to the system. 2. The system is operational and connected to the inventory database and catalog subsystem. 3. Available base teas and raw ingredients are stored and up-to-date in the system database.
Workflow	**Normal Flow (Successful):** 1. The Customer opens the custom tea blend interface. 2. The system **retrieves** and displays all available base teas and raw organic ingredients. 3. The Customer selects a base tea, adds desired ingredients, and adjusts the percentages. 4. The Customer clicks "Save Custom Formula". 5. The system **validates** the availability of all selected ingredients in the requested quantities against the inventory database. 6. The system calculates the total price of the custom blend

	based on the selections. 7. The system generates a unique identifier (ID) for the custom formula. 8. The system saves the custom formula to the Customers profile. 9. The system displays a success message with the calculated price and formula details. **Alternative Flow (Successful - Modify Selections):** • **3A1.** At step 3, the Customer changes the percentages or removes an ingredient before saving. • **4A1.** The system dynamically updates the tentative price • **5A1.** The Customer clicks "Save Custom Formula" and the system proceeds to step 5. **Error Flow (Ingredients Unavailable):** • **5E1.** At step 5, the system identifies that one or more selected ingredients are out of stock. • **6E1.** The system displays an error message highlighting the specific unavailable items. • **7E1.** The custom formula is not saved, and the system prompts the Customer to adjust percentages or choose different ingredients (returns to Step 3).
Post-conditions / Response	**Successful:** A new custom tea formula is uniquely recorded, saved in the database under the Customer's profile, and the system is ready for the Customer to proceed to ordering. **Unsuccessful:** No custom formula is created or saved. The Customers profile remains unchanged, and the system prompts the Customer to modify their selection.
Comments	The system strictly enforces ingredient availability checks during this process, but physical stock is not deducted until an actual order checkout is executed.

UC-18: Execute Order Checkout [Writer: Yasmin Itawi]

Use Case Title	UC-18: Execute Order Checkout
Description	This use case describes the process where a registered customer reviews and confirms their final shopping cart contents, inputs their delivery coordinates, selects a preferred payment method, and submits a binding order request to the system.
Actors	Primary Actor: Customer Secondary Actor: Email Service
Data	Shopping cart items, item quantities, delivery coordinates (Full Address, City, Phone Number), total order amount, payment method selection, Order ID, order status.
Stimulus / Trigger	The Customer clicks the "Proceed to Checkout" button from the active shopping cart interface.
Pre-conditions	1. The Customer is successfully registered and authenticated (logged into their account). 2. The Customer's active shopping cart contains at least one valid item.
Workflow	1. The Customer initiates the checkout process by clicking the checkout action. 2. The system retrieves and displays the final shopping cart contents, including item names, quantities, individual prices, and the initial subtotal amount. 3. The Customer reviews and confirms the cart contents. 4. The system prompts the Customer to enter their delivery coordinates (Full Address, City, and Phone Number) and select a preferred payment method (Cash on Delivery or Credit Card). 5. The Customer enters the required delivery coordinates and selects their payment method. 6. The system validates the completeness and format of the entered delivery coordinates. 7. If any required fields are missing or the phone number format is invalid, 7.1. The system displays a specific validation error message highlighting the incorrect or missing fields. 7.2. The system blocks submission and prompts the Customer to

	correct the information, and the flow returns to step 5. 8. Else (Fields are valid), the system calculates the final total amount, incorporating any applicable shipping fees and previously applied promotional discounts. 9. The Customer confirms the final invoice summary and clicks the "Submit Order" button. 10. If the selected payment method is "Credit Card", 10.1. The system invokes the internal payment processing mechanism (UC-19: Process Financial Transaction). 10.2. If the transaction is declined by the external gateway, 10.2.1. The system executes the exception flow (UC-22: Handle Payment Rejection) and displays a payment failure error message. 10.2.2. The system leaves the transaction unsaved, and the flow returns to step 4. 10.3. Else (Payment is successful), the system proceeds directly to step 11. 11. The system securely creates and saves a new order record in the master database, setting its initial tracking status flag to "Placed". 12. The system updates the inventory database by deducting the corresponding quantities of the purchased tea products. 13. The system clears all active items from the Customer's current digital shopping cart session. 14. The system generates a unique Order ID and displays a success confirmation screen along with the final order summary to the Customer. 15. The system requests the Email Service to automatically transmit a formal order confirmation email containing the invoice and delivery tracking details to the Customer's registered email address.
Post-conditions / Response	On Success: 1. A new order record is successfully stored in the database with the status set to "Placed", linked directly to the Customer's profile. 2. Physical inventory levels for the purchased items are accurately decremented. 3. The Customer's active shopping cart session is completely emptied. 4. A success confirmation screen is rendered, and an automated invoice

	email is dispatched by the Email Service. On Failure: 1. No new order record is created or saved in the database system. 2. Inventory levels and stock volumes remain entirely unchanged. 3. All items remain intact inside the Customer's shopping cart without any data loss. 4. A specific error message is displayed on the UI, and the system state remains at the checkout submission step.
Comments	The checkout execution and database updates must run inside an atomic transaction block; any unexpected system crash or gateway timeout before step 14 must trigger an immediate database rollback to prevent data inconsistency.

UC-19: Process Financial Transaction [Writer: Reema Qashoo]

Use Case Title	UC-19: Process Financial Transaction
Description	A Customer who has confirmed their order at the checkout screen initiates a payment request through the Tea Shop system. The system securely tokenises the payment details and forwards the transaction to an external PaymentGateway for authorisation. The gateway validates the card and returns an approval or rejection response. If approved, the system records the transaction, updates the order status to "Paid", generates a unique payment reference, and triggers a confirmation email via the EmailService. If declined, the system notifies the Customer and allows a retry with different payment details or order cancellation.
Actors	• Customer (Primary Actor): initiates the payment request. • PaymentGateway (Secondary Actor): external API that authorises or rejects the transaction. • EmailService (Secondary Actor): dispatches the payment confirmation email on success.
Data	• Payment method selected by Customer (credit card / debit card / digital wallet). • Tokenised card details (card number, expiry, CVV - never stored by the system). • Order total amount and currency. • PaymentGateway authorisation response code (approval / decline reason). • Generated unique payment reference number. • Order record identifier and current order status. • Customer email address used for confirmation dispatch.
Stimulus / Trigger	User command issued by the Customer upon clicking "Confirm & Pay" on the checkout screen, after UC-18 (Execute Order Checkout) has validated the cart contents and delivery address.
Pre-conditions	1. The Customer account record exists in the system with status Verified and the session is authenticated. 2. An order record exists in the system with status "Pending Payment", linked to the Customer's account, containing at least one item with confirmed stock availability. 3. The system has successfully completed UC-18 (Execute Order Checkout): cart contents are finalised and a valid delivery address is stored against the order record. 4. The Customer has selected a payment method on the checkout screen and the payment form is fully loaded.

Workflow	(The use case starts when the Customer clicks "Confirm & Pay" on the checkout screen.) 1. The Customer reviews the final order total and selects a payment method (credit card, debit card, or digital wallet). 2. The Customer enters card details and clicks "Confirm Payment". 3. The system tokenises the card details and sends a secure authorisation request to the PaymentGateway API. 4. The PaymentGateway validates the card and checks available funds. 5. If authorised (Approved - Normal Path): 5.1 The system records the authorisation code and generates a unique payment reference number. 5.2 The system updates the order record status from "Pending Payment" to "Paid" in the database. 5.3 The system triggers the EmailService to dispatch an order confirmation receipt with the payment reference, order summary, and estimated delivery date. 5.4 The system redirects the Customer to the order confirmation page displaying the payment reference number. 5.5 The system adds the order to the Warehouse Staff pending fulfilment queue (UC-23). 6. Else, if declined (Alternative Path - UC-22: Handle Payment Rejection): 6.1 The PaymentGateway returns a rejection response with a decline reason code (e.g., insufficient funds, invalid card, timeout). 6.2 The system invokes UC-22 and displays a payment failure notification to the Customer with the decline reason. 6.3 The order record status remains "Pending Payment" in the database and no charge is made to the Customer's account. 6.4 If Customer selects "Retry": the sequence flow returns back to step 1 to allow entering different card details. 6.5 Else, if Customer selects "Cancel Order": the system updates the order status to "Cancelled" in the database, releases all reserved stock quantities back to active inventory, and returns the Customer to the shopping cart view. 7. Else, if PaymentGateway connection fails (Error Path 1): 7.1 The system fails to reach the PaymentGateway API due to a network or connectivity error. 7.2 The system logs the network failure with a timestamp and

	displays a technical connectivity error message to the Customer. 7.3 The order record status is strictly maintained as "Pending Payment" and no transaction log is created. 7.4 The system prompts the Customer to retry after a short interval or contact Customer Support. 8. Else, if database update fails after approval (Error Path 2): 8.1 The PaymentGateway has successfully approved the transaction, but the system encounters a critical database error while saving the record. 8.2 The system logs the database inconsistency and flags the order record status as "Pending Verification" to prevent duplicate charges. 8.3 The system records the payment reference number and displays an instruction notifying the Customer to contact Customer Support for manual confirmation. 8.4 The system automatically escalates the transaction metadata to the FinanceOfficer queue for manual resolution (UC-35).
Post-conditions / Response	Successful Payment Flow: 1. The order record status is updated to "Paid" in the database. 2. A unique payment reference number is securely stored and linked to the target order record. 3. The transaction is fully recorded in the payment audit log with a timestamp, amount, and gateway authorisation code. 4. The order entity is visible in the Warehouse Staff pending fulfilment queue. 5. A transaction receipt email has been successfully dispatched to the Customer via the EmailService. Unsuccessful Declined Flow: 1. The order record status remains strictly in the "Pending Payment" state within the database. 2. The specific PaymentGateway rejection code is appended to the order record history for audit purposes. 3. No monetary charge or transaction log entry is created against the Customer's financial account. Unsuccessful Cancelled by Customer: 1. The order record status is updated to "Cancelled" in the database.

	2. Any previously reserved item quantities for this order are fully released back to the active inventory count. Unsuccessful Error Path 2 (DB Failure after Approval): 1. The order record status is set to "Pending Verification" in the database to halt automatic workflows. 2. A high-priority system discrepancy log entry is created and the task is pushed to the FinanceOfficer active dashboard for manual reconciliation.
Relationships	• <<include>> from UC-18 (Execute Order Checkout): UC-19 is always invoked as a mandatory sub-step of the checkout process. • <<extend>> UC-22 (Handle Payment Rejection): conditionally triggered when the PaymentGateway returns a declined response code.
Comments	The system must never store raw card data at any point; all card information is handled exclusively through tokenisation via the PaymentGateway. The system stores only the payment reference number and authorisation status. In Error Path 2, the FinanceOfficer must manually verify the transaction against gateway records to prevent duplicate charges. The CustomerSupportRep may initiate a refund via the PaymentGateway integration if a return is subsequently approved (UC-34, UC-36).

2.7 Activity Diagram

- **Lead:** Yasmin Itawi
- **Contributors:** Suhaib Ewidat (Discussing &Reviewing), Reema Qashoo (Discussing &Finalising)

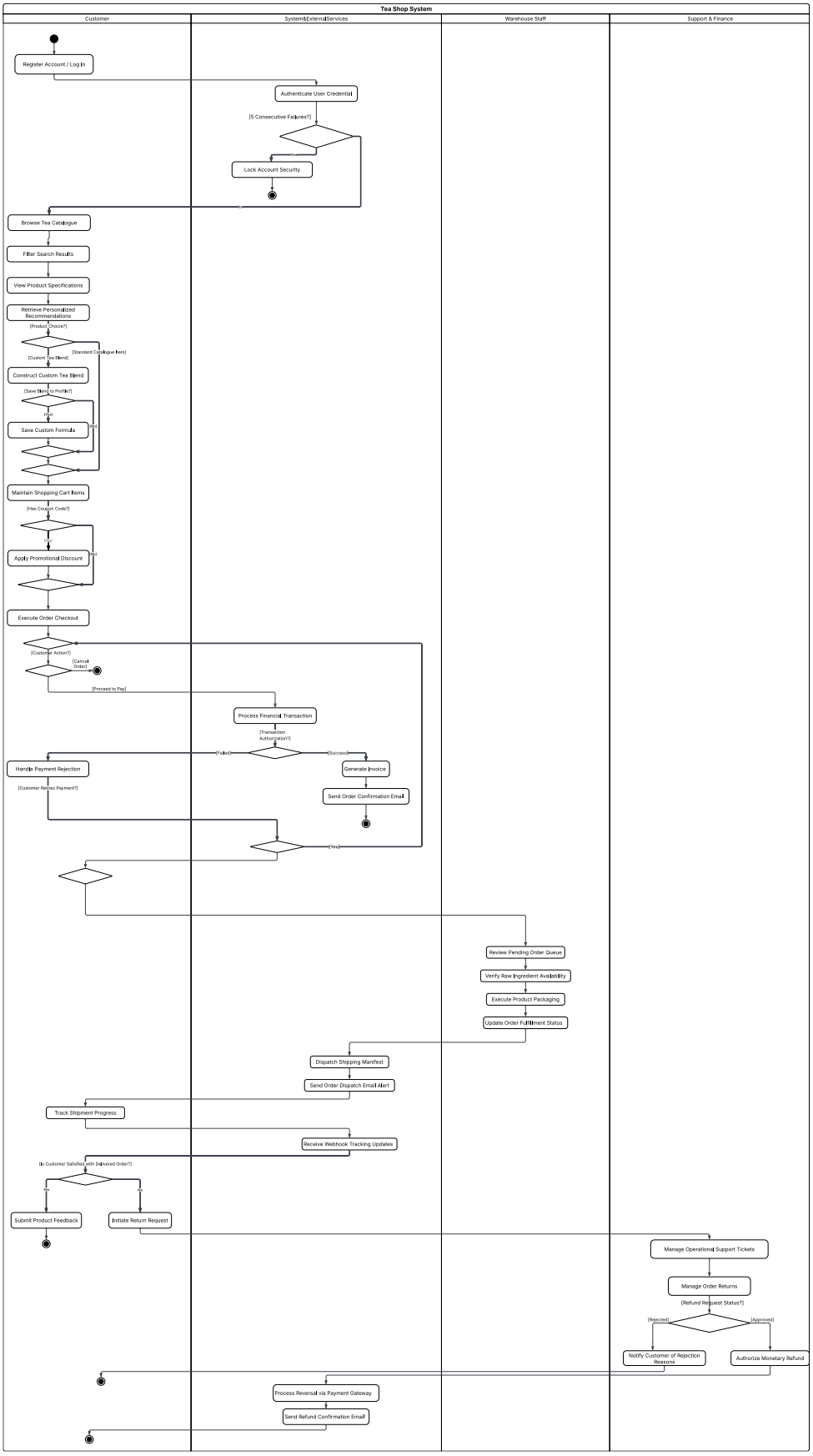


Figure 2:Overall Activity Diagram

3. System Analysis and Modelling

3.1 System CLASS Diagrams

Analysis Class Diagram

- **Lead:** Suhaib Ewidat
- **Contributors:** Reema Qashoo (Discussing &Reviewing), Yasmin Itawi (Discussing &Finalising)

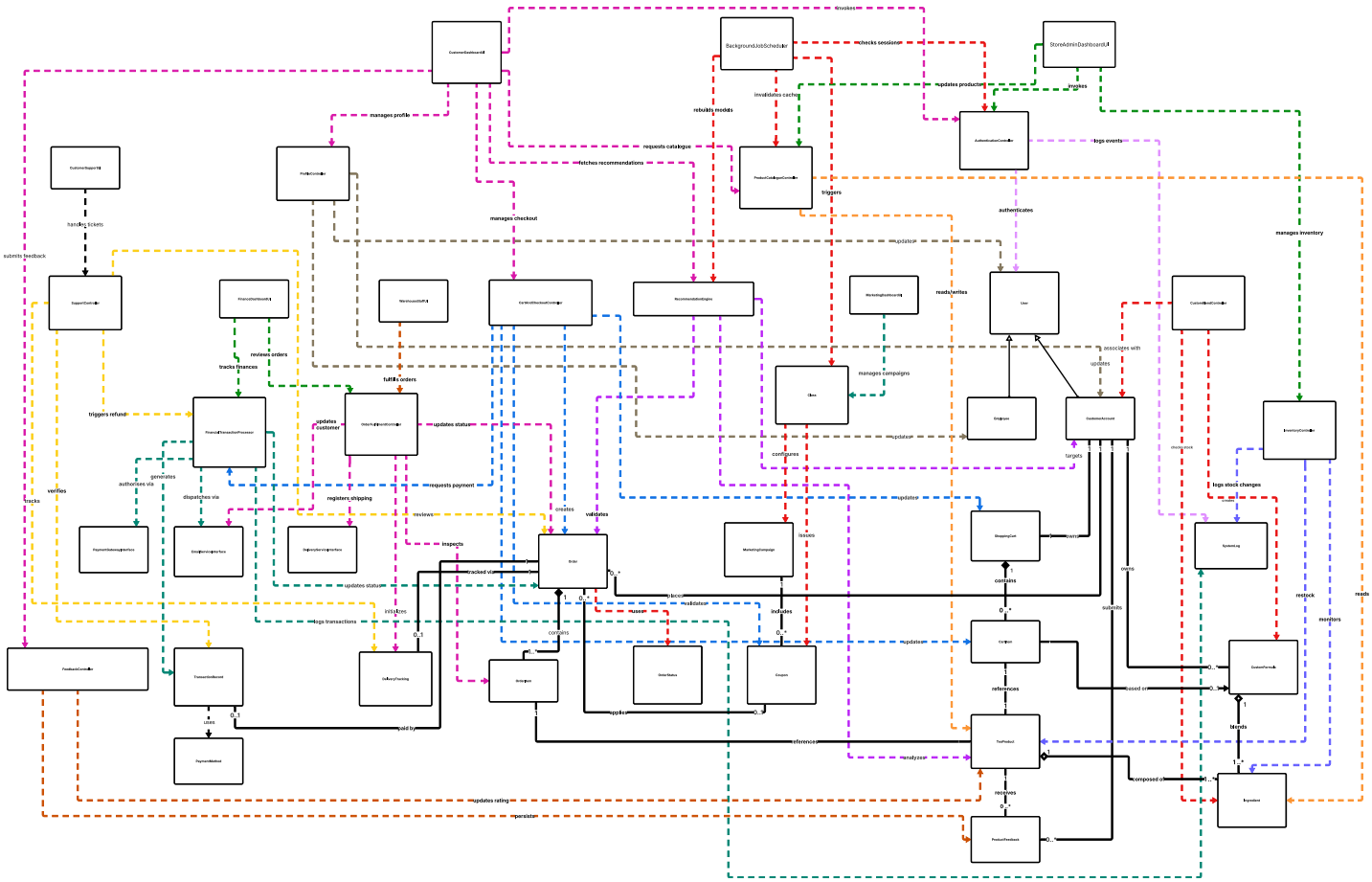


Figure 3: Analysis Class Model

Detailed Class Diagram

- **Lead:** Yasmin Itawi
- **Contributors:** Suhaib Ewidat(Discussing&Reviewing), Reema Qashoo(Writing&Finalising)

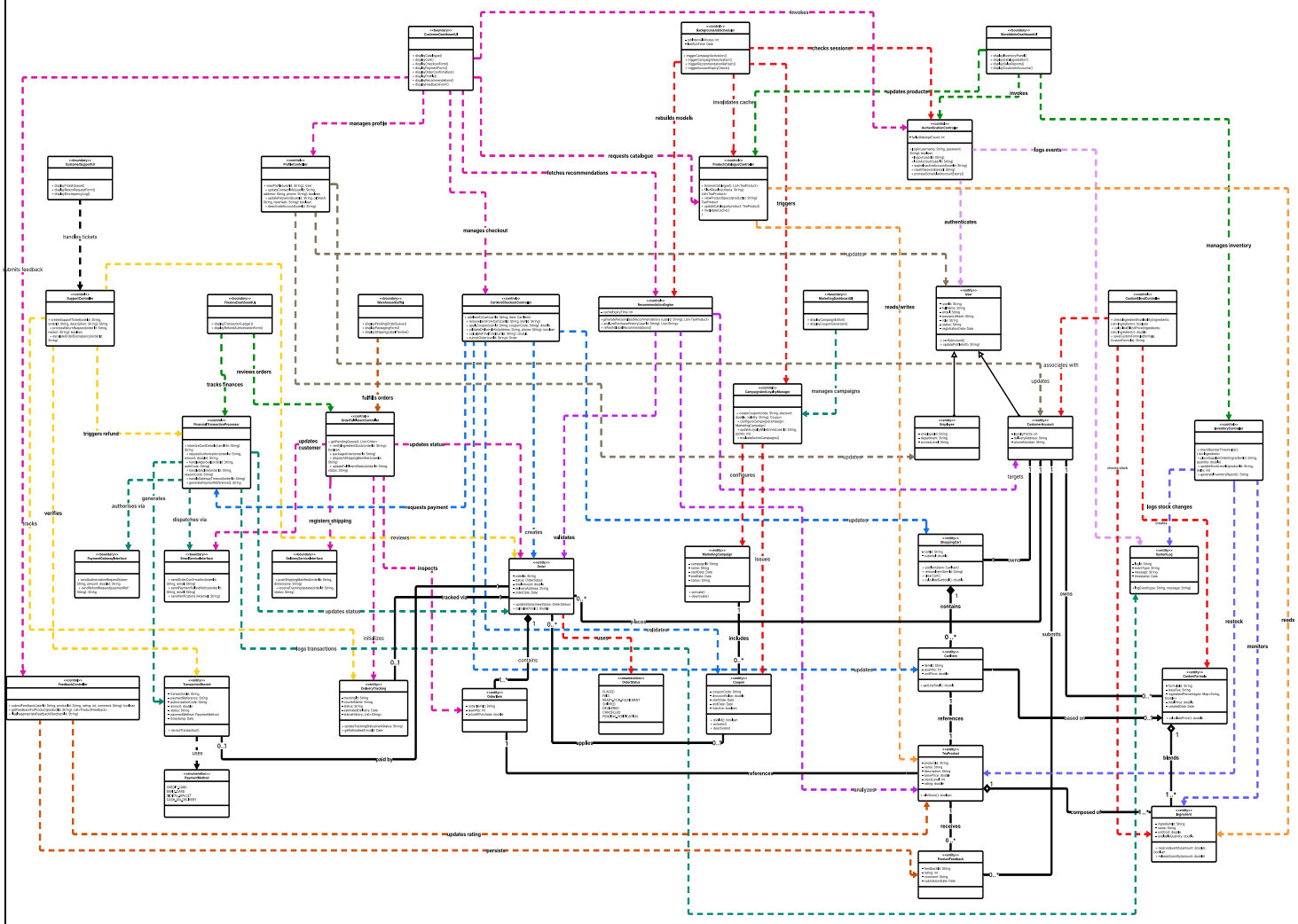


Figure 4: Detailed Class Model

3.2 System Sequence modelling and Analysis

3.2.1 UC-12: Construct Custom Tea Blend [Writer: Suhaib Ewidat]

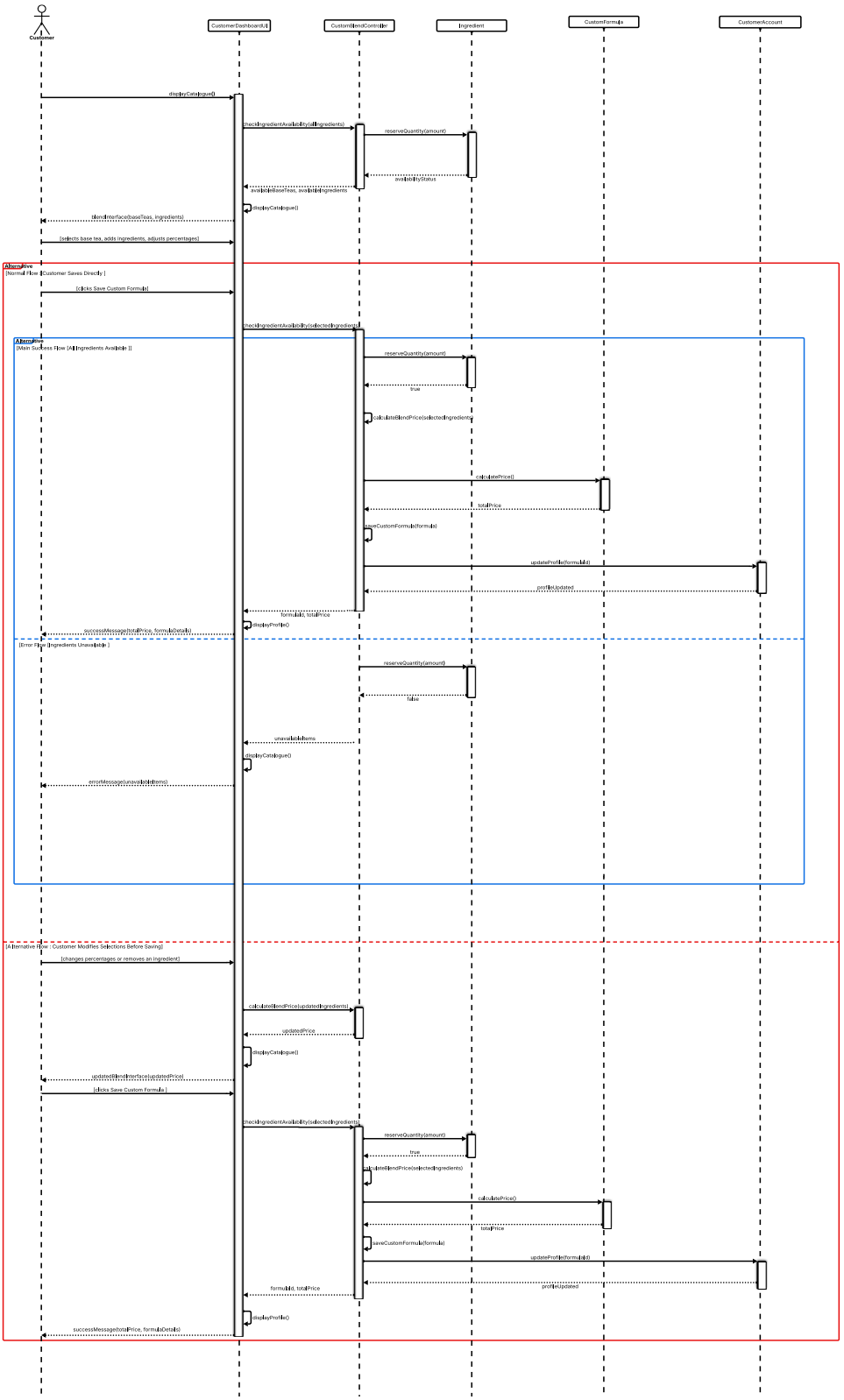


Figure 5: Sequence Diagram [Suhaib]

3.2.2 UC-18: Execute Order Checkout [Writer: Yasmin Itawi]

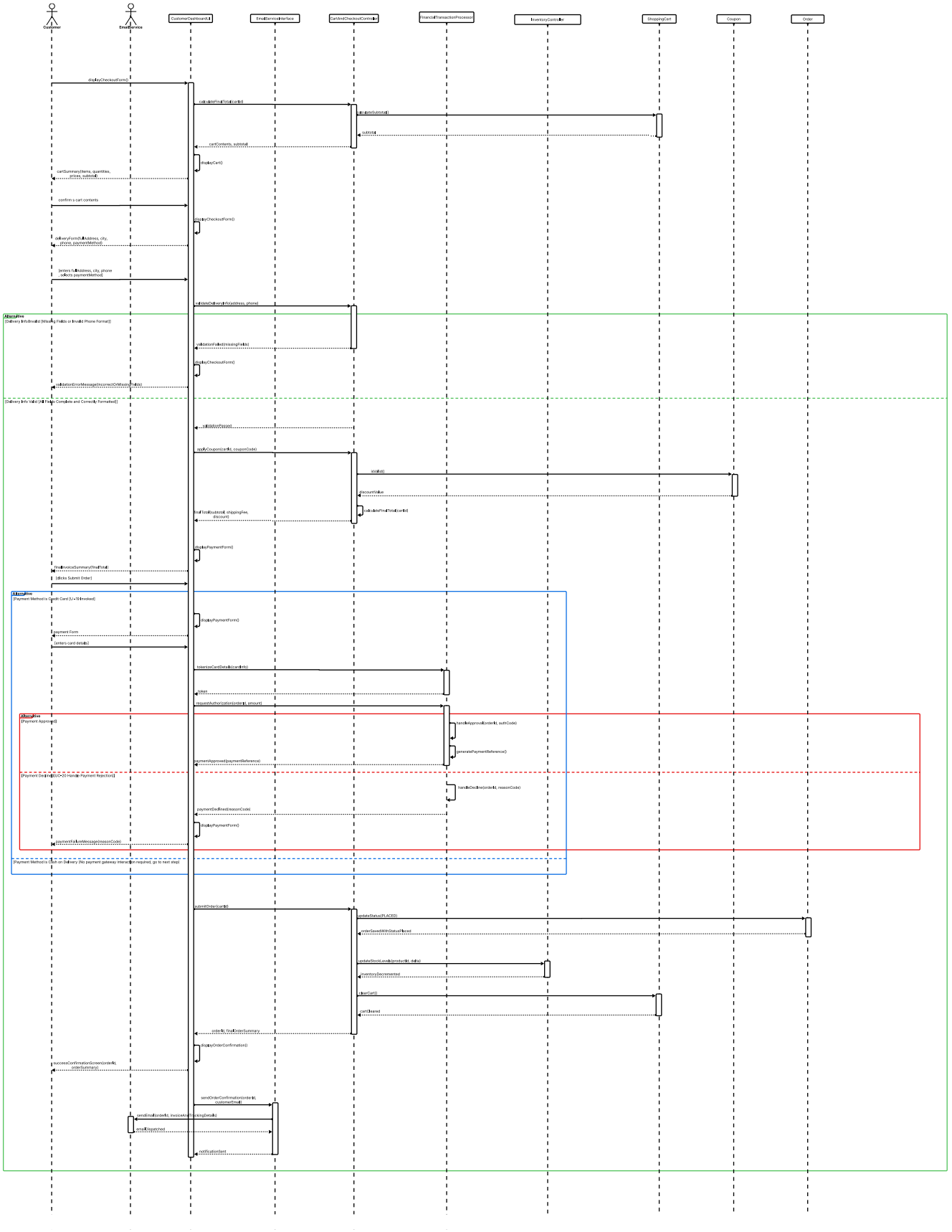


Figure 6: Sequence Diagram [Yasmin]

3.2.3 UC-19: Process Financial Transaction [Writer: Reema Qashoo]

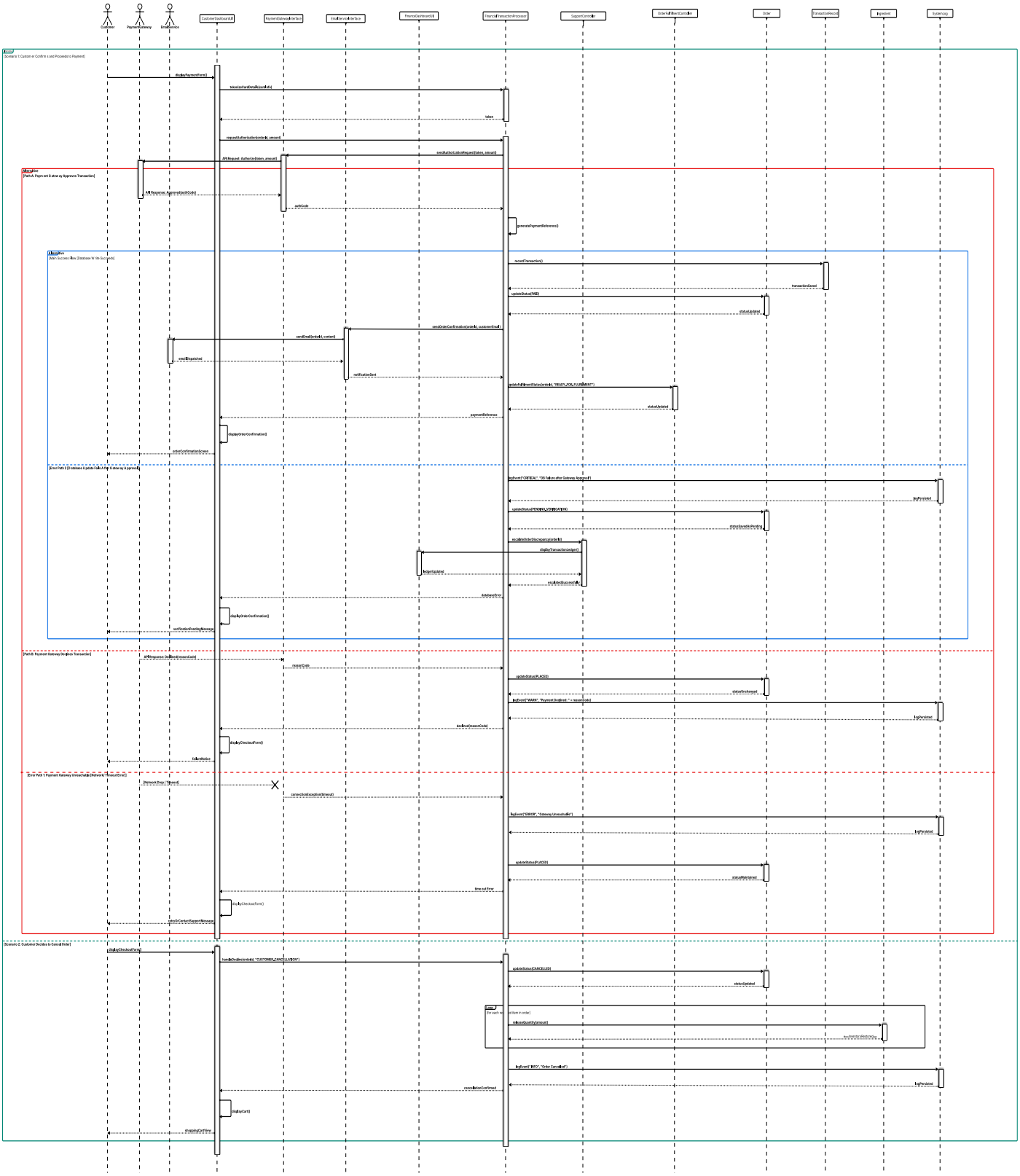


Figure 7: Sequence Diagram [Reema]

4. System Design and Modelling

4.1 System Design Goals

- **Lead:** Reema Qashoo
- **Contributors:** Yasmin Itawi (Discussing &Reviewing), Suhaib Ewidat (Discussing &Finalising)

4.1.1 Low Coupling and High Cohesion

- **Design Purpose:**

The system is designed using the Boundary-Control-Entity (BCE) pattern to ensure that each component has a single, clear responsibility (High Cohesion) and that components do not rely heavily on each other (Low Coupling). This makes the system easier to update, test, and maintain.

- This goal is directly addressed and visible in our **Detailed Class Diagram**.
- **Design Choices Supporting This Goal:**

1. **Separation of Concerns (BCE):** We separated the system into three distinct layers. For example, presentation tasks are isolated inside boundary classes like [CustomerDashboardUI], business logic is handled by controllers like [CartAndCheckoutController], and data storage is kept inside entities like [Order] and [ShoppingCart].
2. **Function-Oriented Components:** Each class focuses only on its main job. The [CustomBlendController] only handles ingredient mixing logic, while all payment operations are completely isolated inside the [FinancialTransactionProcessor] control class.
3. **Strict Interaction Boundaries:** Boundary classes never talk directly to Entity classes. They must pass through Control classes (such as [ProductCatalogueController]), which reduces direct dependencies across the system.

4.1.2 Extensibility and Modularity

- **Design Purpose:**

The system must be open for expansion but closed for modification. We want to ensure that changing or upgrading external services (like payment or email vendors) will not affect our core business system.

- This goal is addressed in the **Detailed Class Diagram** through the use of dedicated wrapper interface boundary classes.
- **Design Choices Supporting This Goal:**

1. **Interface-Based Abstraction:** Core controllers do not connect directly to external systems. Instead, they interact with abstraction boundaries like [PaymentGatewayInterface], [EmailServiceInterface], and [DeliveryServiceInterface].

2. **Plug-and-Play Third-Party Services:** If the shop decides to change the payment provider (for example, moving from credit cards to a new digital wallet), developers only need to update the concrete implementation inside [PaymentGatewayInterface]. The core financial controller [FinancialTransactionProcessor] will remain completely untouche

4.1.3 Specific System Design Goal: Transactional Data Integrity and Reliability

- **Design Purpose:**

Since the system processes online financial transactions for custom tea formulas, a critical requirement is to guarantee that no system crash or database failure leaves a customer's money in an inconsistent state.

- This goal is addressed using specific state logic in the **Detailed Class Diagram** and fault-handling logic in the **Checkout/Payment Sequence Diagrams (e.g., UC-19)**.
- **Design Choices Supporting This Goal:**

1. **Robust Order Status Tracking:** In the **Detailed Class Diagram**, we created a specific enumeration called [OrderStatus]. It includes fallback states like [PENDING_VERIFICATION] and [CANCELLED].

2. **Database Failure Interception:** As modeled in our payment sequences, if the external payment gateway successfully approves a charge via [sendAuthorizationRequest()] but our local system fails to save the order due to a database error, the [FinancialTransactionProcessor] catches this issue immediately.

3. **Operational Safety Logs:** Instead of failing silently, the system updates the order status to [PENDING_VERIFICATION], creates a secure [TransactionRecord], and calls [SystemLog.logEvent()]. This pushes a high-priority ticket to the [FinanceDashboardUI] so warehouse staff and finance officers can manually check the payment and prevent double charging the customer.

4.2 Component Diagram

- **Lead:** Suhaib Ewidat
- **Contributors:** Reema Qashoo (Discussing &Reviewing), Yasmin Itawi (Discussing &Finalising)

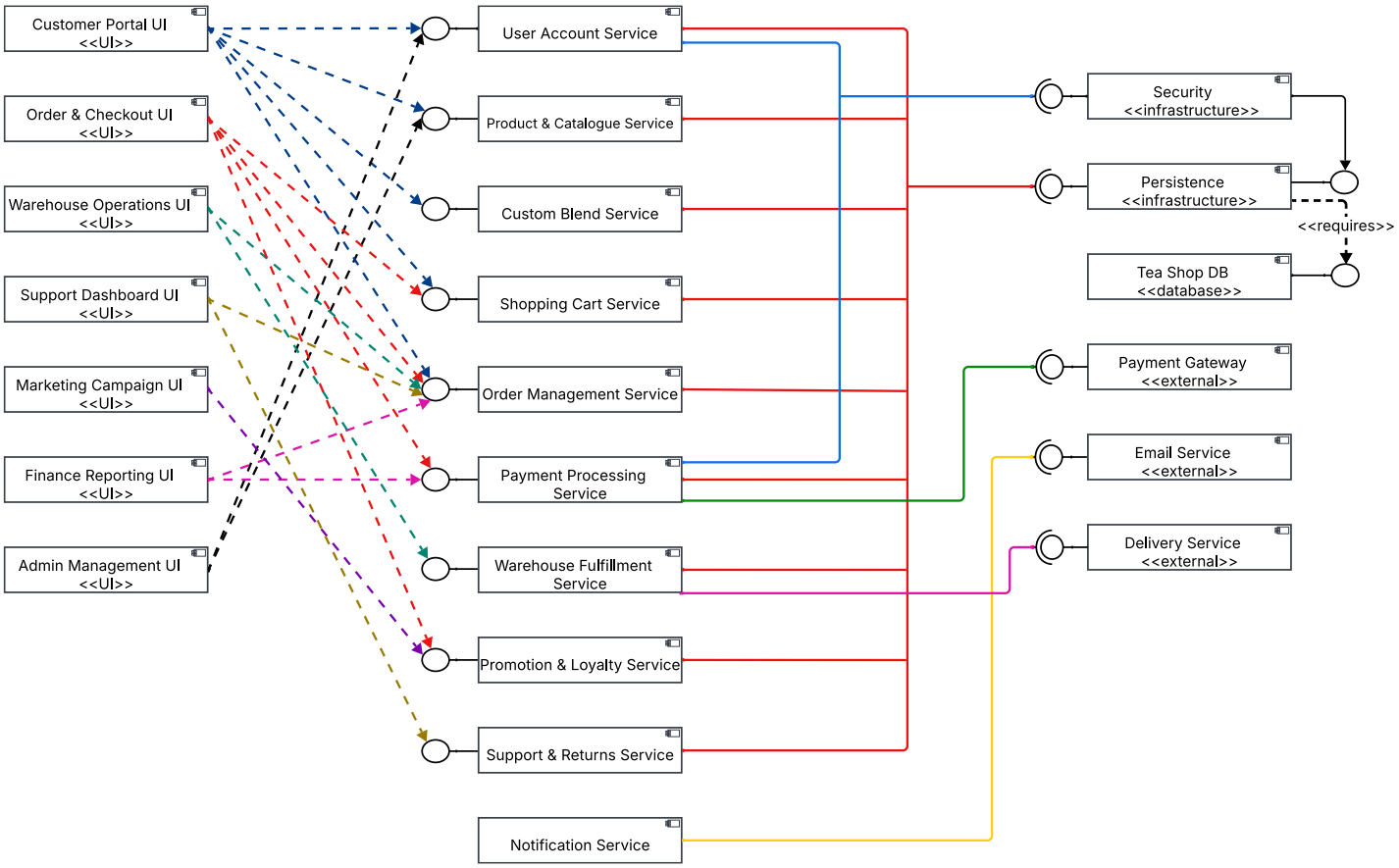


Figure 8: Component Diagram

4.3 Overall Architecture Diagram

- **Lead:** Yasmin Itawi
- **Contributors:** Suhaib Ewidat(Discussing&Reviewing), Reema Qashoo(Writing&Finalising)

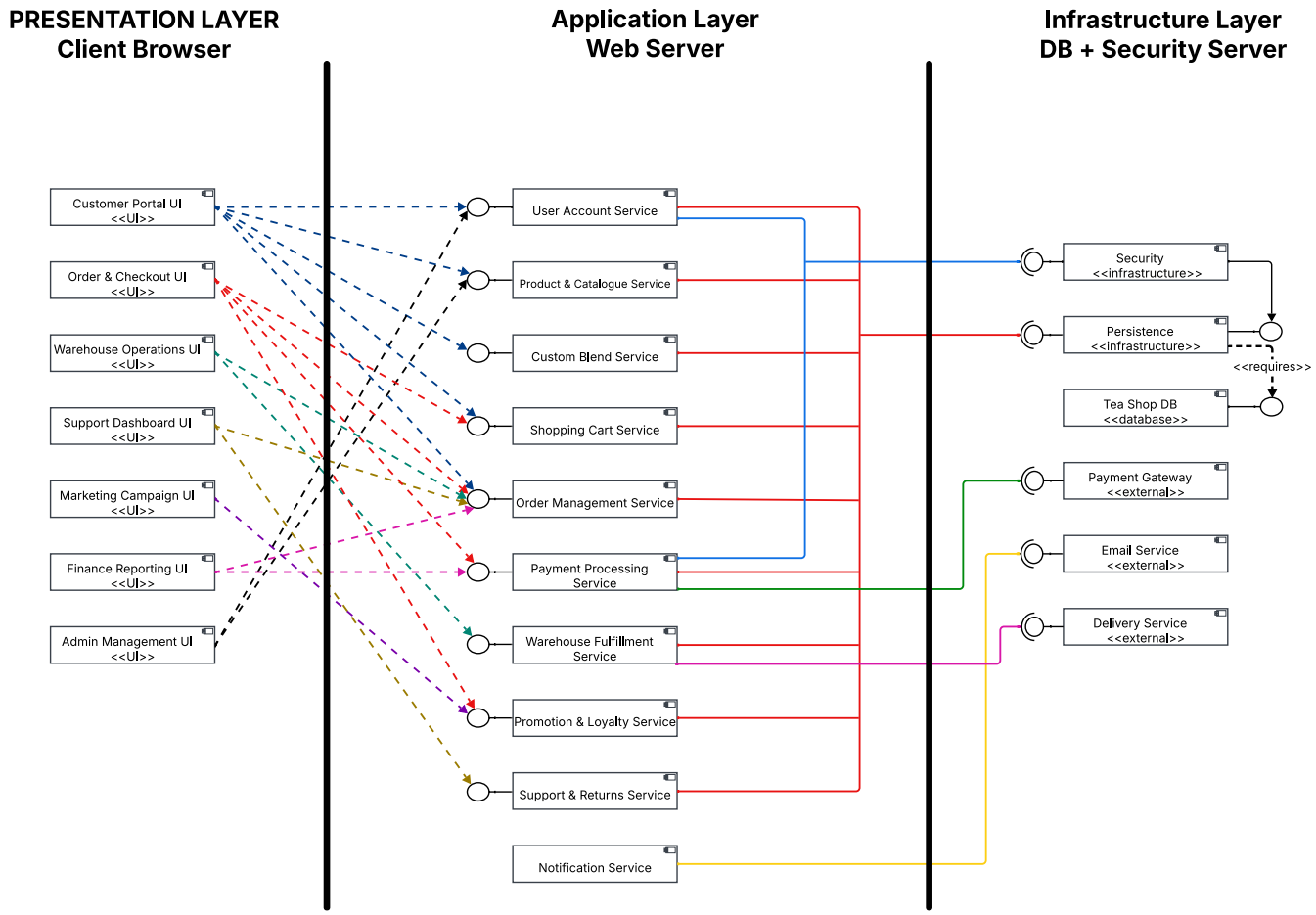


Figure 9: Overall Architecture Diagram

4.3.1 Architecture Justification

For the Digital Tea Shop System, we chose a **3-Tier Layered Architecture** built on a **Client-Server model**. The system is divided into three distinct, stacked layers: the **Presentation Layer** (Boundaries), the **Business Logic Layer** (Controllers), and the **Data Layer** (Entities). The client side (the browser or dashboard UI) sends requests to the server side (controllers and databases) to process data.

4.3.2 Rationale for Choosing 3-Tier Layered Architecture

1. Achieving Low Coupling and High Cohesion

The layered architecture strictly prevents different parts of the system from mixing together. Each layer has one specific job.

Justification: The **Presentation Layer** only contains UI classes like [CustomerDashboardUI] and [StoreAdminDashboardUI]. It knows nothing about how database queries or payment processing work.

- The **Business Logic Layer** contains controllers like [CartAndCheckoutController] and [CustomBlendController]. This layer acts as the middleman, processing data and enforcing business rules.
- Because layers can only talk to the layer directly below them, we achieve very low coupling. If we want to redesign the user interface entirely, we can do so without touching a single line of backend logic or database code.

2. Supporting Extensibility and Modularity (NFR: Maintainability)

A layered system makes it simple to add new features or swap external components without breaking the existing codebase.

Justification: In a tea shop system, we rely heavily on external third-party services for payments, shipping, and emails. In our architecture, these are isolated at the edge of the Business Logic Layer through interface wrappers like [PaymentGatewayInterface] and [DeliveryServiceInterface].

- If the business decides to switch from the current credit card gateway to a different provider, the change is contained entirely within that specific layer interface. The core ordering system stays exactly the same, making the software highly modular and cheap to maintain over time.

3. Ensuring Transactional Data Integrity (NFR: Reliability & Fault Tolerance)

Separating the business rules from the physical database allows the system to catch errors and handle failures safely before data gets corrupted.

Justification: Financial safety is critical when customers order custom tea blends. When a payment is processed, the interaction happens securely inside the Business Logic Layer via the [FinancialTransactionProcessor].

- If the external banking gateway charges the card but a network crash happens before saving to the database, the intermediate logic layer intercepts this failure. Because the database layer is separate, the controller can safely pause the workflow, tag the internal record as [PENDING_VERIFICATION] in the [Order] entity, and log an alert via [SystemLog]. This keeps our system data accurate and reliable even during critical hardware or runtime faults.

4.4 Deployment Diagram

- **Lead:** Reema Qashoo
- **Contributors:** Yasmin Itawi (Discussing &Reviewing), Suhaib Ewidat (Discussing &Finalising)

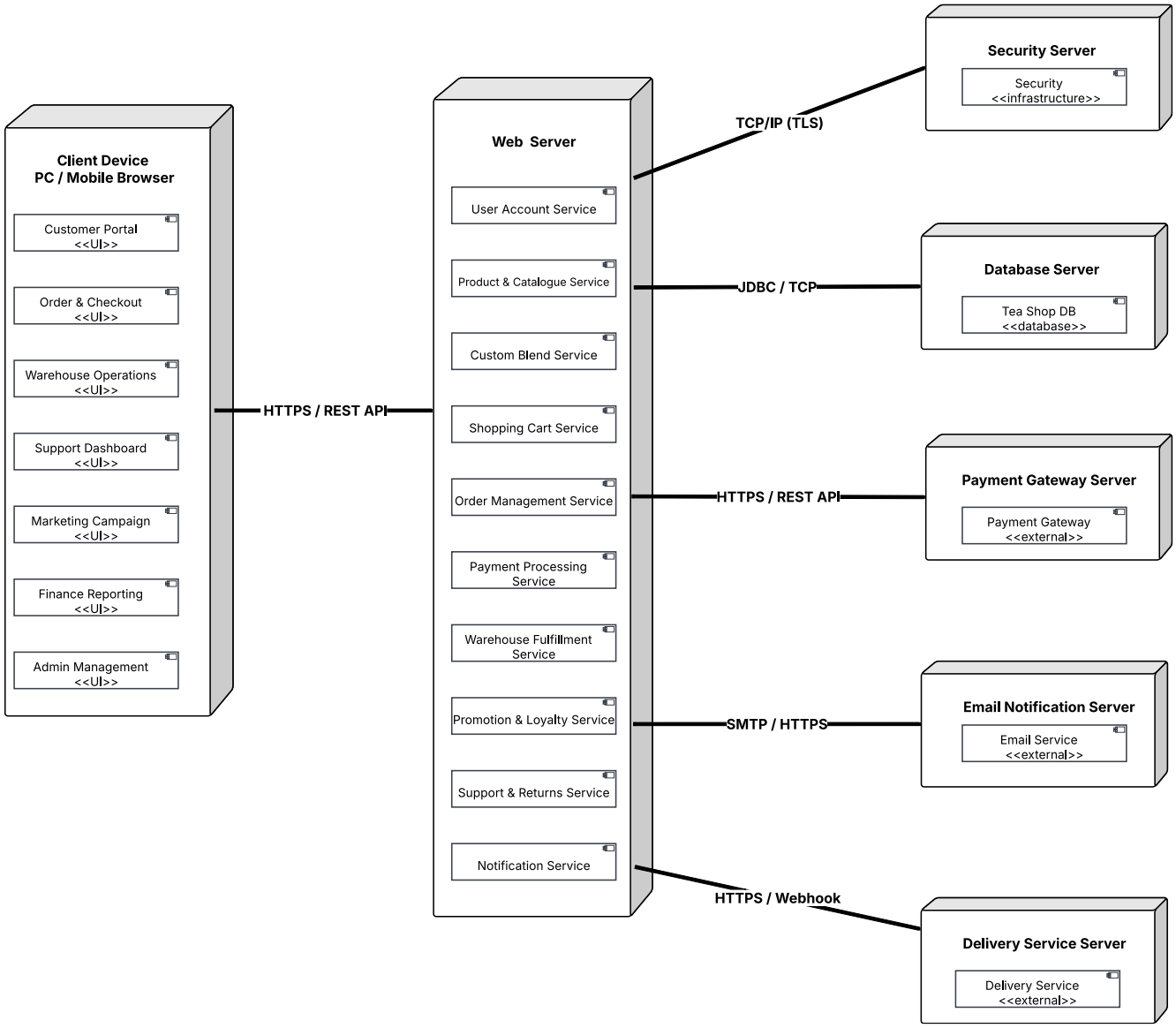


Figure 10: Deployment Diagram

Appendix-I: Project Meetings

Meeting 1 — Internal Team Meeting

Date & Time: Thursday, 16 April 2026, 7:30 PM
Location: Online

Attendees:
Yasmin Itawi, Reema Qashoo, Suhaib Ewidat

Topics Discussed:
• Understanding the assigned Tea Shop project
• Planning team roles and responsibilities
• Discussing project workflow and communication strategy

Decisions / Outcomes:
• Team roles were assigned (Project Manager, Requirement Engineer, Programmer)
• Weekly online meetings were agreed upon
• Decision-making would be based on voting after discussion

Meeting 2 — Customer Meeting

Date & Time: Wednesday, 6 May 2026, 8:00 PM
Location: Online

Attendees:
Developer Group: Yasmin Itawi, Reema Qashoo, Suhaib Ewidat
Customer Group: Mohammad Zaid, Mohammad Abu Hijleh, Layan Seyam, Rawand Radi

Topics Discussed:
• Main idea of the Tea Shop System
• Types of tea products and drinks
• Delivery service requirements
• Payment options

Decisions / Outcomes:
• System will support ready-made tea and custom tea blends
• Delivery tracking will be included
• Cash on delivery and online payment were approved

Requirements Validation:
The customer group confirmed these as core business requirements.

Meeting 3 — Customer Meeting

Date & Time: Sunday, 10 May 2026, 12:00 PM
Location: Online

Attendees:
Developer Group: Yasmin Itawi, Reema Qashoo, Suhaib Ewidat
Customer Group: Mohammad Zaid, Mohammad Abu Hijleh, Layan Seyam, Rawand Radi

Topics Discussed:
• Account registration and login
• Search and filtering options
• Checkout process
• Admin features

Decisions / Outcomes:
• Login/registration confirmed as mandatory
• Search by category and price approved
• Admin will manage products, orders, and reviews

Requirements Validation:
Customer group reviewed and approved the proposed system requirements.

Meeting 4 — Internal Team Meeting

Date & Time: Monday, 1 June 2026, 8:30 PM
Location: Online

Attendees:
Yasmin Itawi, Reema Qashoo, Suhaib Ewidat

Topics Discussed:
• Actor analysis
• Use-case modelling
• Distribution of individual use cases

Decisions / Outcomes:
• Final actors were confirmed
• Main use cases were approved
• Individual use cases were assigned to each member

Meeting 5 — Internal Team Meeting

Date & Time: Thursday, 25 June 2026, 9:00 PM
Location: Online

Attendees:
Yasmin Itawi, Reema Qashoo, Suhaib Ewidat

Topics Discussed:
• Class diagrams and architecture design
• Final report organization
• Reviewing instructor feedback

Decisions / Outcomes:
• Diagram revisions were assigned
• Final report sections were distributed
• Additional meetings scheduled before submission